

Backend aplikacji – dokumentacja developerska

1. Cel dokumentu

Ten dokument opisuje aktualny backend aplikacji, sposób połączenia modułów, główne przepływy danych oraz sposób uruchomienia i testowania projektu lokalnie. Ma służyć kolejnym programistom jako szybkie wejście do kodu, tak aby mogli bezpiecznie rozwijać backend i podłączać frontend.

2. Stack technologiczny

- Python
- Flask
- Flask-SQLAlchemy
- Flask-JWT-Extended
- Marshmallow
- SQLite na potrzeby lokalnego developmentu
- pytest do testów

Aktualnie projekt działa na **SQLite**, co upraszcza lokalne uruchomienie. Jest to rozwiązanie developerskie. Docelowo architektura nadal nadaje się do późniejszego powrotu do PostgreSQL.

3. Struktura katalogów

Najważniejsze elementy backendu:

```
backend/  
|-- app/  
| |-- __init__.py  
| |-- config.py  
| |-- extensions.py  
| |-- seed_data.py  
| |-- api/  
| |-- models/  
| |-- repositories/  
| |-- schemas/  
| |-- services/  
| |-- utils/  
| '-- cli/  
|-- tests/  
|-- requirements.txt  
'-- run.py
```

3.1. Odpowiedzialność warstw

app/api/

Warstwa HTTP. Zawiera blueprinty Flask i endpointy REST. Odpowiada za:

- odbiór requestu,
- walidację danych wejściowych przez schemat,
- wywołanie odpowiedniego serwisu,
- zwrócenie spójnej odpowiedzi JSON.

Route'y powinny być cienkie. Logika biznesowa nie powinna trafiać tutaj.

app/services/

Główna logika biznesowa systemu. Tu realizowane są:

- rejestracja i logowanie,
- wyszukiwanie usług i terapeutów,
- liczenie dostępności,
- rezerwacja i anulowanie wizyt,
- obsługa opinii,
- operacje admin/staff.

To najważniejsza warstwa do dalszej rozbudowy.

app/repositories/

Warstwa dostępu do danych. Zawiera zapytania do modeli i bazy. W projekcie repozytoria są używane częściowo — część serwisów korzysta też bezpośrednio z modeli ORM.

app/models/

Modele SQLAlchemy, czyli definicje tabel i relacji.

app/schemas/

Schematy Marshmallow do:

- walidacji requestów,
- serializacji odpowiedzi.

app/utils/

Wspólne helpery:

- błędy,
- role,
- paginacja,
- funkcje czasu,
- auth helpers.

app/seed_data.py

Dane startowe do developmentu i testów.

tests/

Testy backendu.

4. Główne moduły backendu

4.1. Auth

Pliki:

- `app/api/auth_routes.py`
- `app/services/auth_service.py`

Obsługuje:

- rejestrację,
- logowanie,
- refresh token,
- logout,
- pobieranie aktualnego użytkownika.

4.2. Katalog usług i terapeutów

Pliki:

- `app/api/services_routes.py`
- `app/api/therapists_routes.py`
- `app/services/catalog_service.py`

Obsługuje:

- listę usług,
- szczegóły usługi,
- terapeutów przypisanych do usługi,
- publiczny profil terapeuty.

4.3. Dostępność

Pliki:

- `app/api/availability_routes.py`
- `app/services/availability_service.py`

Obsługuje liczenie wolnych terminów na podstawie:

- reguł grafiku,

- wyjątków,
- istniejących wizyt,
- czasu trwania usługi.

Sloty nie są trzymane w tabeli. Są generowane dynamicznie.

4.4. Wizyty

Pliki:

- `app/api/appointments_routes.py`
- `app/services/appointment_service.py`

Obsługuje:

- utworzenie rezerwacji,
- listę wizyt pacjenta,
- szczegóły wizyty,
- anulowanie wizyty.

4.5. Historia konsultacji

Pliki:

- `app/api/consultations_routes.py`
- `app/services/consultation_service.py`

Obsługuje:

- historię zakończonych wizyt pacjenta,
- opis konsultacji.

4.6. Opinie

Pliki:

- `app/api/reviews_routes.py`
- `app/services/review_service.py`

Obsługuje:

- publiczne opinie terapeuty,
- dodanie opinii po zakończonej wizycie,
- przeliczanie średniej oceny terapeuty.

4.7. Admin

Pliki:

- `app/api/admin_services_routes.py`
- `app/api/admin_therapists_routes.py`
- `app/api/admin_availability_routes.py`
- `app/services/admin_service.py`
- `app/services/therapist_admin_service.py`
- `app/services/schedule_admin_service.py`

Obsługuje:

- zarządzanie usługami,
- tworzenie i edycję terapeutów,
- przypisywanie usług do terapeutów,
- zarządzanie grafikami i wyjątkami.

4.8. Staff

Pliki:

- `app/api/staff_appointments_routes.py`
- `app/services/staff_appointment_service.py`

Obsługuje:

- listę wizyt do obsługi,
- zmianę statusów wizyt,
- dodanie lub edycję opisu konsultacji.

5. Modele i relacje

Najważniejsze encje:

- `User`
- `Role`
- `UserRole`
- `TherapistProfile`
- `Service`
- `TherapistService`
- `AvailabilityRule`
- `AvailabilityException`

- Appointment
- ConsultationSummary
- Review
- EmailNotification

5.1. Najważniejsze relacje

- użytkownik może mieć wiele ról,
- terapeuta jest powiązany z użytkownikiem 1:1,
- terapeuta może realizować wiele usług,
- usługa może być realizowana przez wielu terapeutów,
- terapeuta ma wiele reguł grafiku i wyjątków,
- pacjent ma wiele wizyt,
- wizyta należy do jednego pacjenta, jednej usługi i jednego terapeuty,
- wizyta może mieć jedno podsumowanie konsultacji,
- wizyta może mieć jedną opinię.

5.2. Snapshoty w wizytach

W tabeli Appointment przechowywane są snapshoty:

- nazwy usługi,
- opisu usługi,
- długości wizyty,
- ceny,
- imienia i nazwiska terapeuty,
- tytułu terapeuty.

Dzięki temu historia wizyt nie zmienia się po edycji katalogu usług lub profilu terapeuty.

6. Role i uprawnienia

Role:

- PATIENT
- THERAPIST
- ADMIN

6.1. Pacjent

Może:

- się zarejestrować,
- zalogować,
- przeglądać usługi i terapeutów,
- sprawdzać dostępność,
- rezerwować wizytę,
- anulować własną wizytę,
- przeglądać własne wizyty i konsultacje,
- dodawać opinię po zakończonej wizycie.

6.2. Terapeuta

Może:

- przeglądać staffowe wizyty przypisane do siebie,
- zmieniać statusy swoich wizyt,
- dodawać opis konsultacji do obsługiwanych wizyt.

6.3. Admin

Może:

- zarządzać usługami,
- zarządzać terapeutami,
- przypisywać usługi,
- zarządzać grafikami,
- przeglądać i edytować staffowe wizyty.

7. Przepływ requestu

Typowy przepływ wygląda tak:

1. request trafia do endpointu Flask w `app/api/...`,
2. dane wejściowe są walidowane przez schemat w `app/schemas/...`,
3. route wywołuje serwis z `app/services/...`,
4. serwis wykonuje logikę biznesową,
5. w razie potrzeby serwis korzysta z repozytorium lub modeli SQLAlchemy,
6. wynik wraca do route,
7. odpowiedź jest opakowana do formatu:
 - `data`,
 - opcjonalnie `meta`,
 - albo `error`.

7.1. Przykład: POST /api/v1/appointments

- route: appointments_routes.py
- schema: AppointmentCreateSchema
- service: AppointmentService.create(...)

Serwis:

- sprawdza usługę,
- sprawdza terapeutę,
- sprawdza powiązanie terapeuty-usługa,
- sprawdza, czy termin istnieje w dostępności,
- sprawdza konflikty,
- zapisuje wizytę,
- tworzy powiadomienia email.

8. Format odpowiedzi API

8.1. Sukces

```
{
  "data": {}
}
```

8.2. Sukces z paginacją

```
{
  "data": [],
  "meta": {
    "page": 1,
    "pageSize": 10,
    "total": 100
  }
}
```

8.3. Błąd

```
{
  "error": {
    "code": "APPOINTMENT_SLOT_NOT_AVAILABLE",
    "message": "Wybrany termin nie jest dostpny."
  }
}
```

Walidacja może zwracać też **details**.

9. Dostępne endpointy

9.1. Public / patient

Auth

- POST /api/v1/auth/register
- POST /api/v1/auth/login
- POST /api/v1/auth/refresh
- POST /api/v1/auth/logout
- GET /api/v1/users/me

Katalog

- GET /api/v1/services
- GET /api/v1/services/<service_id>
- GET /api/v1/services/<service_id>/therapists
- GET /api/v1/therapists/<therapist_id>
- GET /api/v1/therapists/<therapist_id>/reviews

Dostępność

- GET /api/v1/availability

Wizyty

- POST /api/v1/appointments
- GET /api/v1/appointments/me
- GET /api/v1/appointments/<appointment_id>
- POST /api/v1/appointments/<appointment_id>/cancel

Konsultacje i opinie

- GET /api/v1/consultations/me
- POST /api/v1/appointments/<appointment_id>/review

9.2. Admin

- GET /api/v1/admin/services
- POST /api/v1/admin/services
- PATCH /api/v1/admin/services/<service_id>
- GET /api/v1/admin/therapists
- POST /api/v1/admin/therapists

- PATCH /api/v1/admin/therapists/<therapist_id>
- POST /api/v1/admin/therapists/<therapist_id>/services
- DELETE /api/v1/admin/therapists/<therapist_id>/services/<service_id>
- GET /api/v1/admin/therapists/<therapist_id>/availability-rules
- POST /api/v1/admin/therapists/<therapist_id>/availability-rules
- PATCH /api/v1/admin/availability-rules/<rule_id>
- DELETE /api/v1/admin/availability-rules/<rule_id>
- GET /api/v1/admin/therapists/<therapist_id>/availability-exceptions
- POST /api/v1/admin/therapists/<therapist_id>/availability-exceptions

9.3. Staff

- GET /api/v1/staff/appointments
- PATCH /api/v1/staff/appointments/<appointment_id>/status
- PUT /api/v1/staff/appointments/<appointment_id>/consultation-summary

9.4. Techniczne

- GET /api/v1/health

10. Jak uruchomić projekt lokalnie

10.1. 1. Wejdź do katalogu backend

```
cd backend
```

10.2. 2. Utwórz i aktywuj virtualenv

macOS / Linux:

```
python3 -m venv .venv
source .venv/bin/activate
```

10.3. 3. Zainstaluj zależności

```
pip install -r requirements.txt
```

10.4. 4. Ustaw zmienne środowiskowe

Utwórz plik `.env` albo ustaw zmienne w shellu.

Przykład:

```
FLASK_ENV=development
DATABASE_URL=sqlite:///app.db
SECRET_KEY=change-me
JWT_SECRET_KEY=change-me-too
CORS_ORIGINS=http://localhost:3000
APP_TIMEZONE=Europe/Warsaw
MAIL_SENDER=noreply@example.com
MAIL_BACKEND=console
```

10.5. 5. Usuń starą bazę, jeśli chcesz startować od zera

```
rm -f app.db
rm -f instance/app.db
```

10.6. 6. Uruchom backend

```
python run.py
```

Co robi `run.py`

- tworzy aplikację Flask,
- wykonuje `db.create_all()`,
- uruchamia `seed_database()`,
- startuje serwer developerski.

To rozwiązanie jest wygodne lokalnie. Na późniejszym etapie warto przejść na migracje.

11. Dane testowe seed

Przykładowe konta:

- admin: `admin@example.com` / `Admin123!`
- terapeuta: `anna@example.com` / `Password123!`
- terapeuta: `piotr@example.com` / `Password123!`
- pacjent: `jan@example.com` / `Password123!`
- pacjent: `ola@example.com` / `Password123!`

W seedzie są też:

- usługi,
- profile terapeutów,
- przypisania usług,

- grafiki,
- przykładowe wizyty,
- przykładowa opinia,
- powiadomienia email.

12. Jak sprawdzić, czy backend działa

12.1. 1. Healthcheck

Wejdź w przeglądarce albo użyj curl:

```
curl http://127.0.0.1:5000/api/v1/health
```

Oczekiwany wynik:

```
{
  "data": {
    "status": "ok"
  }
}
```

12.2. 2. Lista usług

```
curl http://127.0.0.1:5000/api/v1/services
```

Powinieneś dostać listę usług.

12.3. 3. Logowanie

```
curl -X POST http://127.0.0.1:5000/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{
  "email": "jan@example.com",
  "password": "Password123!"
}'
```

W odpowiedzi dostaniesz token.

12.4. 4. Autoryzowany request

Podstaw accessToken do:

```
curl http://127.0.0.1:5000/api/v1/users/me \
-H "Authorization: Bearer <accessToken>"
```

13. Przykładowy manualny test przepływu pacjenta

1. POST /auth/login
2. GET /services
3. pobierz service_id
4. GET /services/<service_id>/therapists

5. pobierz `therapist_id`
6. GET `/availability?serviceId=...&therapistId=...&from=...&to=...`
7. wybierz `startAt`
8. POST `/appointments`
9. GET `/appointments/me`
10. POST `/appointments/<id>/cancel`

To jest podstawowy flow, który powinien działać przed podpinaniem frontendu.

14. Jak uruchomić testy

W katalogu backend:

```
pytest tests -q
```

14.1. Co testujemy

Testy obejmują:

- auth,
- katalog,
- dostępność,
- wizyty,
- review/admin/staff.

14.2. Jak działają testy

- testy używają konfiguracji `testing`,
- baza jest tworzona w pamięci,
- dane seedujące są ładowane automatycznie w `fixture`.

15. Najważniejsze miejsca do dalszej rozbudowy

15.1. Backend

Najczęstsze miejsca rozwoju:

- nowe endpointy w `app/api/`,
- nowa logika w `app/services/`,
- nowe walidacje w `app/schemas/`,
- rozszerzenia modeli w `app/models/`,
- dodatkowe testy w `tests/`.

15.2. Frontend

Frontend powinien korzystać głównie z tych endpointów:

- auth,
- services,
- therapists,
- availability,
- appointments,
- consultations,
- reviews.

Najpierw warto podłączyć:

1. login/register,
2. listę usług,
3. terapeutów,
4. dostępność,
5. rezerwację,
6. moje wizyty.

16. Ważne uwagi techniczne

16.1. 1. SQLite jest tymczasowe

Dobre do developmentu lokalnego, ale nie jest docelowym rozwiązaniem dla rezerwacji terminów na większą skalę.

16.2. 2. db.create_all() jest developerskie

Obecnie wygodne, ale później powinno zostać zastąpione przez migracje.

16.3. 3. Logika biznesowa powinna trafiać do serwisów

Nie rozbudowuj route'ów o skomplikowaną logikę.

16.4. 4. Seed jest częścią rozwoju lokalnego

Jeśli dodajesz nowe moduły, zadбай o aktualizację `seed_data.py`, żeby inni mogli łatwo uruchomić projekt.

16.5. 5. Testy trzeba rozwijać razem z backendem

Każdy nowy endpoint biznesowy powinien dostać test.

17. Rekomendowany sposób pracy nad backendem

Przy dodawaniu nowej funkcjonalności najlepiej iść w tej kolejności:

1. doprecyzować model danych,
2. dodać lub poprawić schemat wejścia/wyjścia,
3. zaimplementować logikę w serwisie,
4. wystawić endpoint w `api/`,
5. dodać testy,
6. zaktualizować seed, jeśli potrzeba.

To pozwala zachować porządek i przewidywalność kodu.

18. Krótka lista startowa dla nowego programisty

Jeśli wchodzisz do projektu po raz pierwszy:

1. sklonuj repo,
2. przejdź do `backend/`,
3. utwórz `virtualenv`,
4. zainstaluj zależności,
5. ustaw `.env`,
6. usuń starą bazę SQLite,
7. uruchom `python run.py`,
8. sprawdź `/api/v1/health`,
9. zaloguj się kontem z seeda,
10. przejdź podstawowy flow pacjenta,
11. uruchom `pytest tests -q`.

Po tych krokach powinieneś mieć działające lokalne środowisko i rozumieć podstawowy przepływ aplikacji.